

System Design Crash Course

Part 1: Fundamentals - Complete Detailed Notes

Every concept explained with an example and a diagram

#	Chapter	Core question it answers
1	What is System Design and Why It Matters	Why do simple systems break?
2	Client-Server Architecture (2-tier and 3-tier)	How do clients and servers talk?
3	Monolith vs Microservices	One big unit or many small ones?
4	Synchronous vs Asynchronous Communication	Wait for the reply or move on?
5	Load Balancer	How is traffic spread across servers?
6	Caching	How do we make repeated requests faster?
7	CDN (Content Delivery Network)	How do we serve far-away users quickly?
8	SQL vs NoSQL (and ACID)	Which database fits which job?
9	Vertical vs Horizontal Scaling	How do we add capacity?
10	Database Sharding	How do we split a huge database?
11	Big Picture, Cheat Sheet and Interview Q&A	How does it all connect?

1. What is System Design and Why is it Important?

1.1 Definition

System design is the process of defining the **architecture** (how components are arranged), the **data flow** (how information moves between them) and the **modules** (the individual pieces) of a system so that it can **handle growing user needs effectively**.

EXAMPLE

A login system built in the simplest possible way works great for **10 users**. Nobody notices any problem. The *same* design with **1 million users** would be overwhelmed and crash. System design is the discipline of planning ahead so this does not happen.

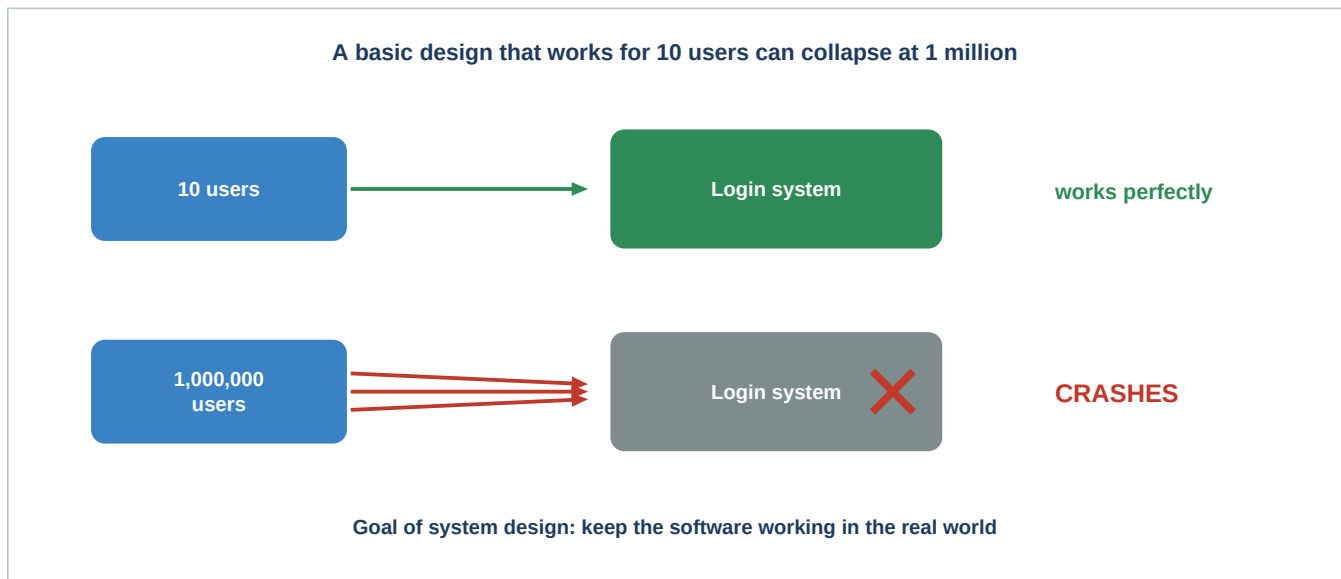


Figure 1: A design that is fine at small scale can collapse at large scale

1.2 Problem 1: Scale (too many users)

A basic design has limited capacity. When traffic grows from 10 to 1,000,000 users, the single server runs out of CPU, memory or connections and the system **crashes** for everybody.

1.3 Problem 2: Latency (users far from the server)

Latency is the **delay between sending a request and receiving the response**. The farther the user is from the server, the larger the delay.

EXAMPLE

A user in the USA accesses a server located on another continent. The page loads, but each request takes about **5 ms** (illustrative). If the same system had a server in the USA, the delay could be about **1 ms**. Nothing is 'broken', but the experience is slower. Multiplied across thousands of requests per page, this becomes very noticeable.

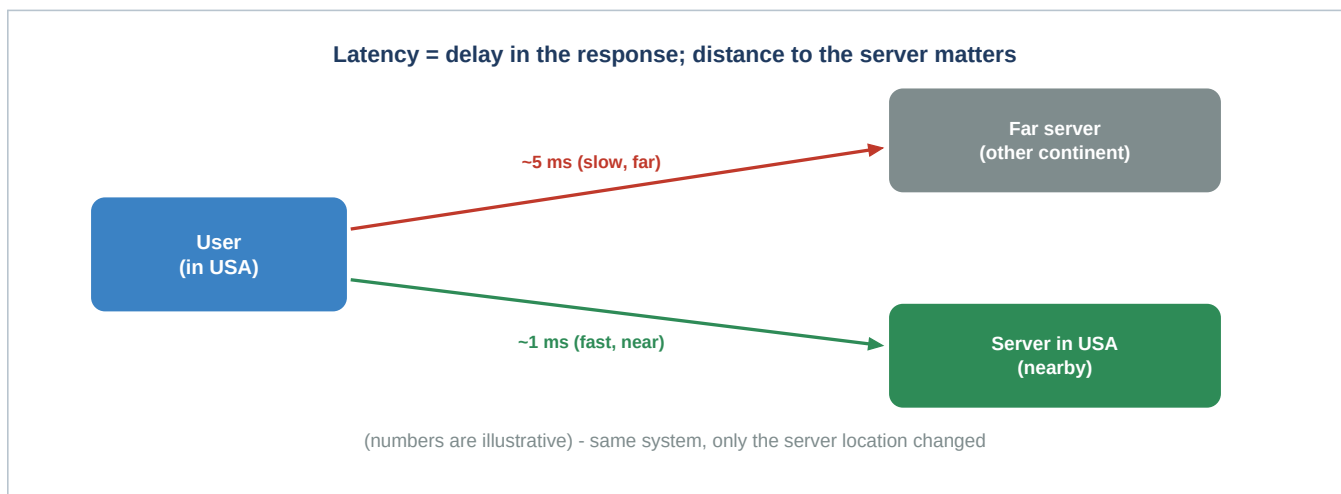


Figure 2: Same system, different server location, very different latency

1.4 The goal of system design

KEY TAKEAWAY

System design makes sure software **does not break when it meets the real world** (large traffic, global users, failures). Every concept in this course (load balancers, caches, CDNs, sharding...) is a tool that makes a system **resilient** in real-world conditions.

2. Client-Server Architecture

2.1 The foundation of the web

Client-server architecture is the **foundation of how the internet works**. A **client** (browser, mobile app) sends a **request**; a **server** processes it and sends back a **response**.

EXAMPLE

You open a food-delivery app and tap 'Restaurants near me'. The app (client) sends a request to the company's server. The server finds restaurants and responds with a list that the app displays.

2.2 HTTP vs HTTPS

- **HTTP is not secure**: data travels as readable text. It is acceptable only for **local testing** or tiny internal setups with a handful of trusted users.
- **HTTPS** adds encryption (TLS). In the real world **HTTPS is always preferred**.

EXAMPLE

On your laptop you run your project at `http://localhost:3000` while developing: fine. When you deploy it to real users, it must be served at `https://yourapp.com`, otherwise passwords and cards could be sniffed on public Wi-Fi.

2.3 Golden rule: the client never talks directly to the database

Direct client-to-database access is **dangerous**. Instead the **server acts as a middleman**: the client asks the server, and the server runs the database queries. This keeps data secure because the server can add logic: authentication, validation, permission checks.

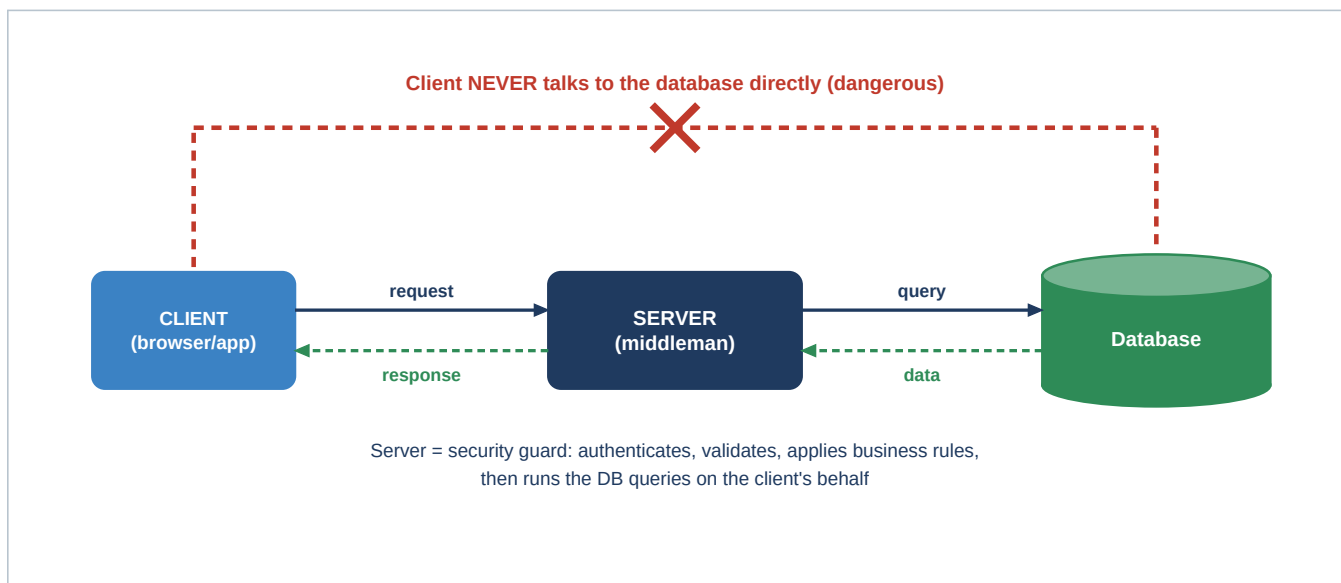


Figure 3: The server sits between client and database as a security layer

EXAMPLE

A banking app should never connect straight to the accounts database. The app asks the server 'show my balance'. The server verifies who you are, queries *only your* account and returns the balance. If the app talked to the DB directly, a hacker could modify the app to run 'show everyone's balance'.

2.4 Two-tier architecture

Two parties are involved:

- **Tier 1 - Client layer:** holds the **presentation logic** (what the user sees) and may also hold the **business logic**.
- **Tier 2 - Server layer:** holds the **data access logic** and connects to the database.

EXAMPLE

A small desktop inventory program for a local shop: the app on the shop computer shows screens and calculates totals itself, and talks to one database server. Simple to build and fine for a small app.

Limitation: logic is spread into the client, so it is harder to scale, secure and update when usage grows.

2.5 Three-tier architecture (the modern standard)

- **Tier 1 - Client layer:** the topmost layer; **provides the user interface**.
- **Tier 2 - Application server layer:** manages the **business logic**. Includes web servers, application servers and APIs. Talks to both the client layer and the database layer.
- **Tier 3 - Database layer:** ensures data is **safely stored, retrieved and maintained**; handles database connections.

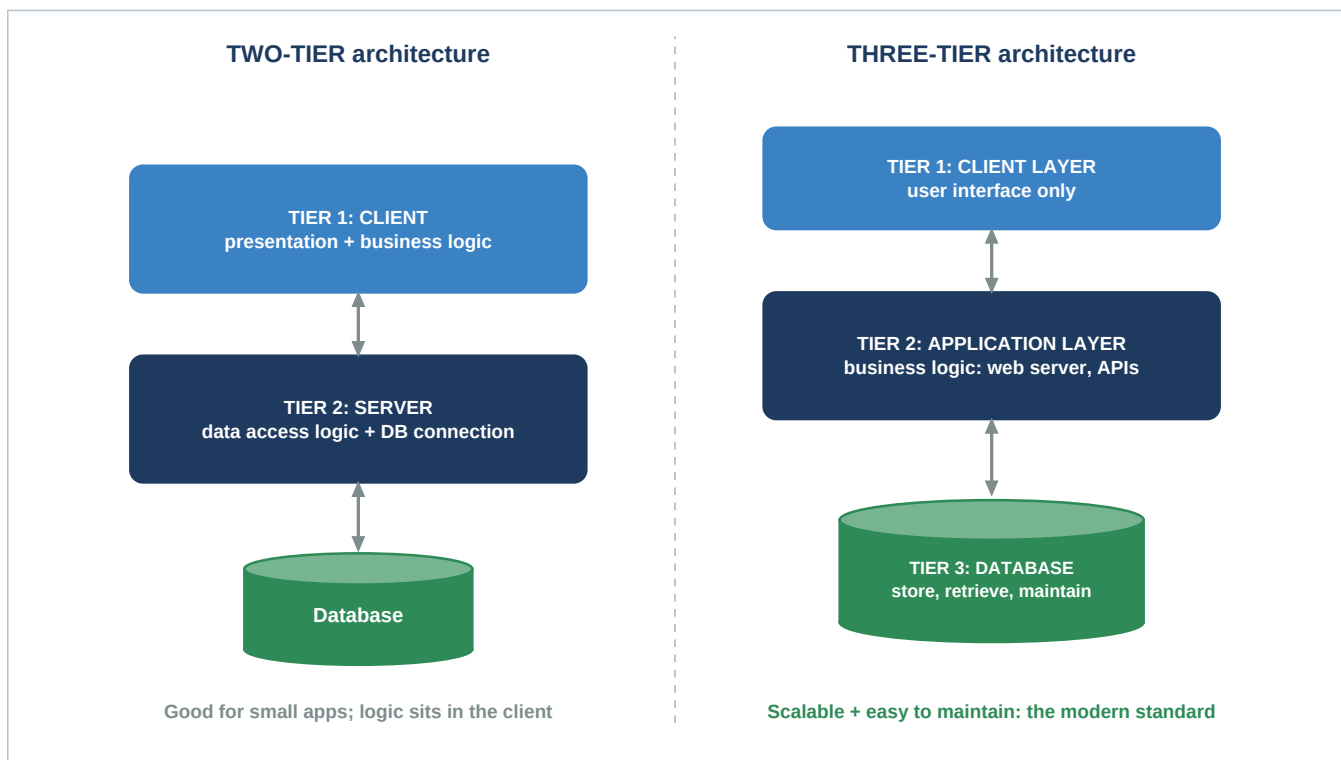


Figure 4: Two-tier vs three-tier architecture

EXAMPLE

Online shopping: the website UI (tier 1) shows the cart. When you click 'Place order', the application server (tier 2) checks stock, calculates discounts and payment rules (business logic). It then asks the database (tier 3) to save the order. Each tier can be scaled or replaced independently.

KEY TAKEAWAY

Three-tier is widely adopted because it is **scalable** (scale each layer separately) and **easy to maintain** (change business rules without touching the UI or the database).

Aspect	Two-tier	Three-tier
Layers	Client + Server	Client + Application server + Database
Business logic lives in	Client (often)	Application server
Good for	Small apps	Real-world, growing apps
Scalability / maintenance	Limited / harder	High / easier

3. Monolithic vs Microservices Architecture

3.1 Monolithic architecture

The **entire application is built as one single unit**: login, payment, dashboard, admin panel, user interface, business logic and data access are written together and **deployed together as one server**, usually with one database.

EXAMPLE

An early-stage e-commerce site where one codebase contains user accounts, product catalogue, payments and notifications. You build, test and deploy the whole thing in one go.

Advantages of a monolith

- **Easy to implement:** one codebase, one project.
- **Easy to test:** run the whole app locally.
- **Fast:** parts call each other inside the same process, with **no network communication**.

Disadvantages of a monolith

- **Hard to scale:** to handle more load in just the payment feature you must scale the entire app.
- **One feature fails, everything is down:** a bug in notifications can crash login and payments too.
- **Slows down as the system grows:** bigger codebase, slower builds, riskier deployments.

3.2 Microservices architecture

The system is broken into **separate small services**. Each service has **one job** (one server, one responsibility), can be **developed, tested and deployed independently**, and has **its own database and business logic**. Services communicate through **APIs**, the most popular style being **REST APIs**.

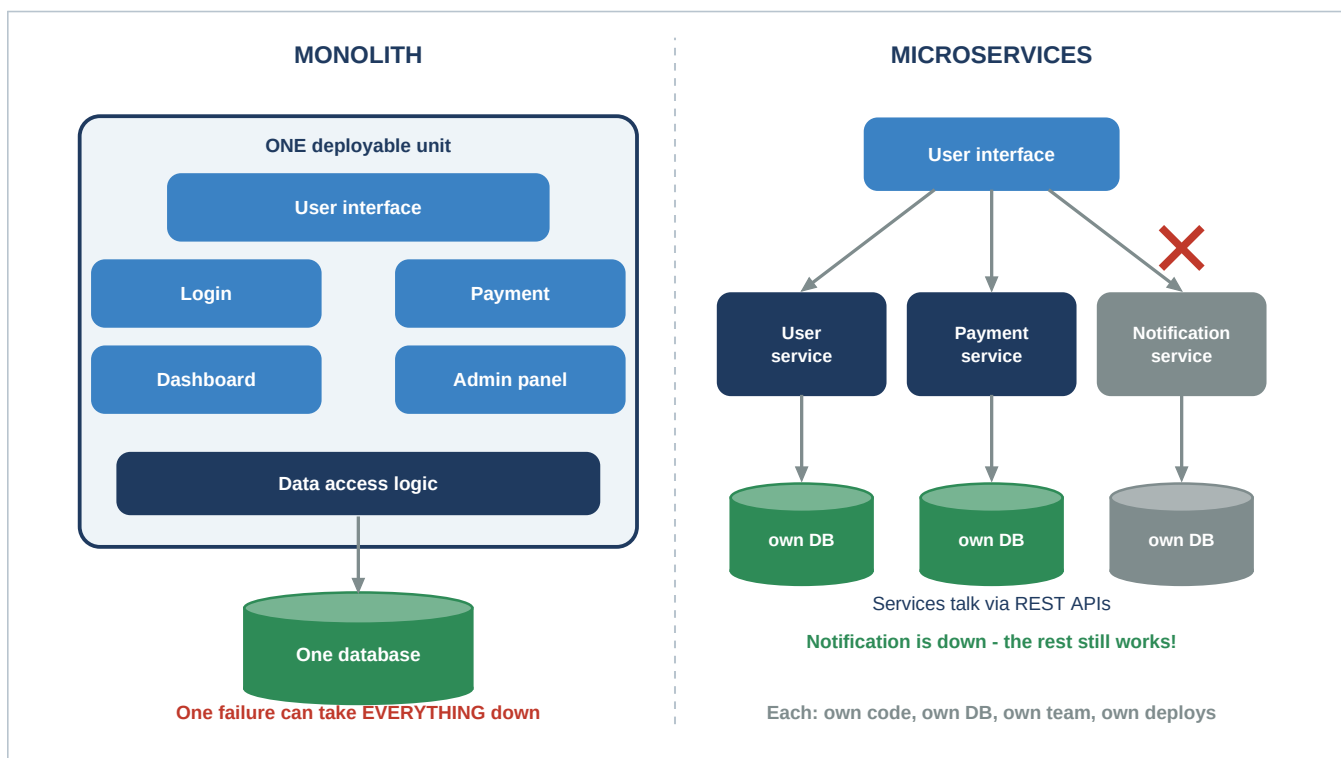


Figure 5: Monolith (one unit, one DB) vs microservices (independent services, each with its own DB)

EXAMPLE

Amazon-like store: one service handles **users**, one handles **payments**, one handles **notifications**. If the notification service goes down, customers can still browse and pay; notifications are simply delayed. The team fixes and redeploys only that service while everything else keeps running. In a monolith the same bug could take the whole website down.

Advantages of microservices

- **Independent parts:** easy to update and scale (scale only the payment service on sale day).
- **Fault isolation:** one service failing does not stop the others.
- **Team autonomy:** separate teams own separate services and work in parallel. Example: the payments team ships a fix on Monday without waiting for the notifications team.

Disadvantages of microservices

- **Complex to manage:** many services, many deployments, many databases.
- **Harder to debug:** one user action may travel through several services (you need tracing and logs).
- **Communication delay:** services talk over the network, so there is always some latency.

Aspect	Monolith	Microservices
Structure	One unit	Many small independent services
Deployment	All together	Each service on its own
Failure impact	One failure can bring everything down	Contained to that service
Scaling	Whole app only	Per service
Speed between parts	Fast (in-process)	Slower (network calls)
Complexity	Low	High
Database	Usually one shared DB	Each service has its own DB
Best for	Starting out, small teams, MVPs	Growing products, many teams

KEY TAKEAWAY

Monolith is **not bad**: it is the right choice when you are just getting started. Move to microservices when you need to grow and scale.

4. Synchronous vs Asynchronous Communication

Services exchange messages in one of two ways. The choice affects the **speed, scalability and complexity** of the system.

4.1 Synchronous communication (blocking)

The sender sends a request and **waits (is blocked)** until the server processes it and sends back the response or acknowledgement. Only then does it continue with its next job. It happens in real time, usually over HTTP(S).

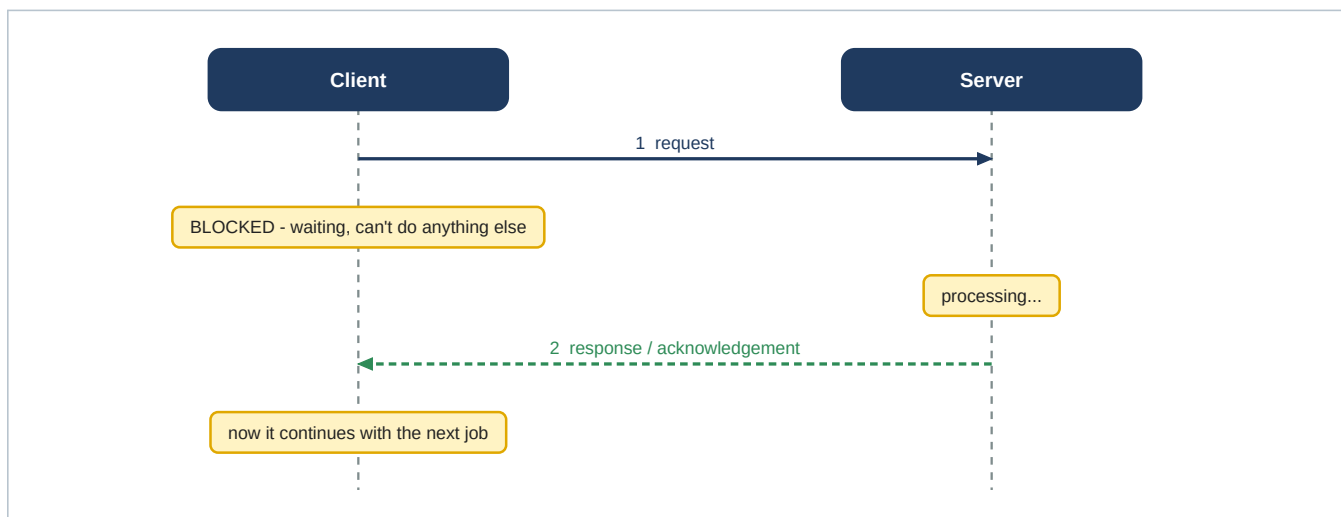


Figure 6: Synchronous: the client is blocked until the response arrives

EXAMPLE

Phone call: you ask a question and wait on the line for the answer before you do anything else. In software: an app calls the payment server and freezes the 'Pay' button until it hears 'success'.

Biggest drawback

If the server is **crashed or too slow**, the client keeps waiting and may eventually **fail or crash**.

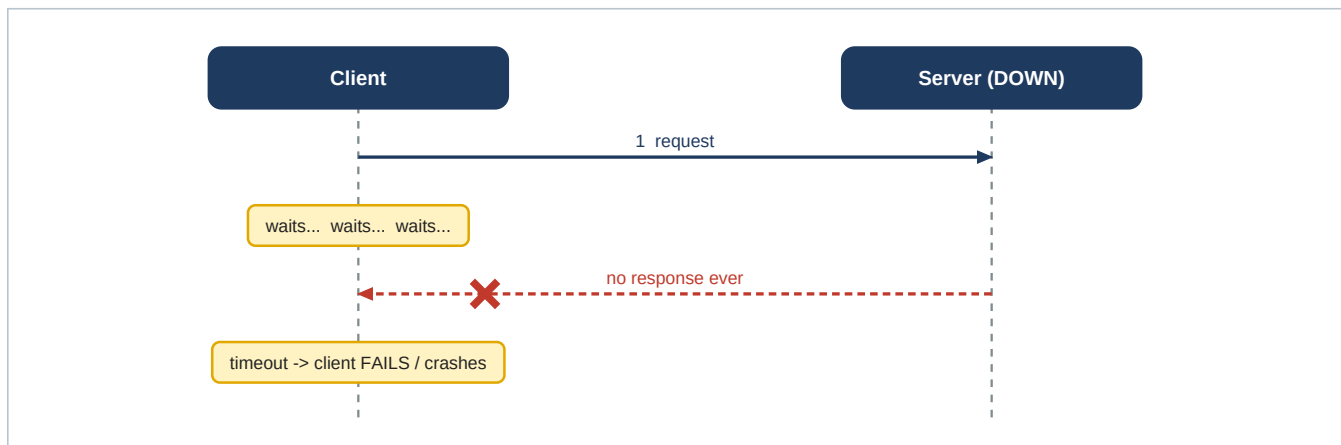


Figure 7: Synchronous call to a dead server: the client waits and then fails

4.2 Asynchronous communication (non-blocking)

The sender **puts a message into a queue and moves on immediately**. The server picks the message up whenever it is ready, processes it in the background, and later sends a **completion event / acknowledgement**. The **queue** is the key component: since many clients send requests, something must store them until the server can process them.

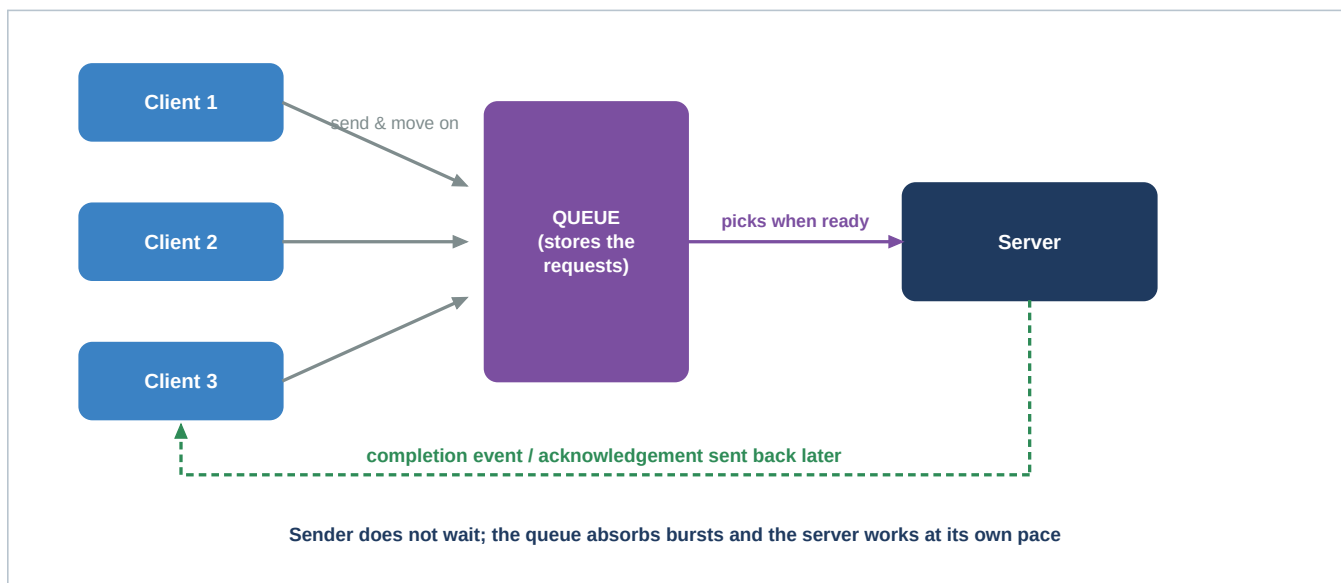


Figure 8: Asynchronous: clients drop requests into a queue; the server processes them later

EXAMPLE

SMS / email: you send it and carry on with your day; the reply arrives later. In software: after you place an order the app instantly says 'Order received'. The order goes into a queue; a worker sends the confirmation email a few seconds later. If the email service is slow, you are not blocked.

Aspect	Synchronous	Asynchronous
Sender behaviour	Waits (blocked)	Moves on immediately
Needs a queue?	No	Yes
If server is down/slow	Client waits or fails	Messages wait safely in the queue
Complexity	Simple	More moving parts
Use when	You need the answer now (login check, payment status)	Work can happen later (emails, image processing, reports)

KEY TAKEAWAY

Neither is 'bad'. Use **synchronous** when the user needs the result immediately and **asynchronous** when the work can be done in the background.

5. Load Balancer

5.1 What is it?

A **load balancer** is a system component that **distributes incoming network traffic across multiple servers** so that no single server gets overwhelmed and there is no single point of failure. It always **sits between the users and the servers** and decides which server gets which request.

5.2 What happens without it, and with it

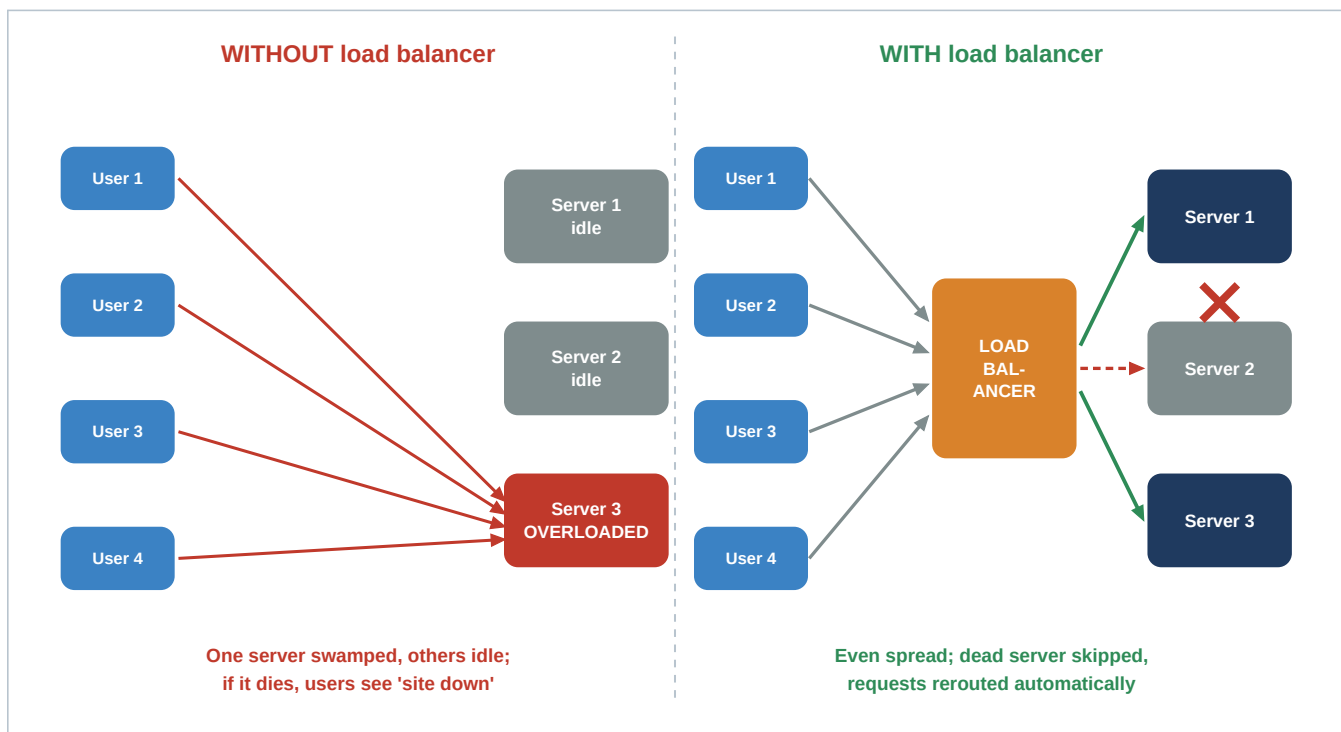


Figure 9: Without a load balancer one server is swamped; with one, traffic is spread and failures are hidden

Without a load balancer, users may all end up on the same server while others sit idle. If that overloaded server crashes, users see 'site is down' even though other servers are perfectly healthy. Users may also reach a distant server when a closer one was available.

With a load balancer, requests are spread out. If a server dies, its traffic is rerouted automatically to the remaining servers using the routing algorithm.

EXAMPLE

A user in the USA is automatically sent to the USA server (less latency). Another user goes to Server 2. Suddenly Server 3 crashes: the users who were going to Server 3 are rerouted to Servers 1 and 2 and never see an error.

5.3 Why load balancers are important

- **High availability:** no single point of failure; the service stays up if one server dies.
- **Traffic optimized:** requests are handled efficiently (for example routed to the nearest server).
- **Easy horizontal scaling:** add a Server 4 and the load balancer starts using it immediately.
- **Handles sudden spikes:** Example: on a normal day an e-commerce site gets 1 million users; on 'Big Billion Day' sale it gets 10 million. Without a load balancer a single server would crash; with one the load is shared evenly across many servers.

5.4 Static vs dynamic load balancers

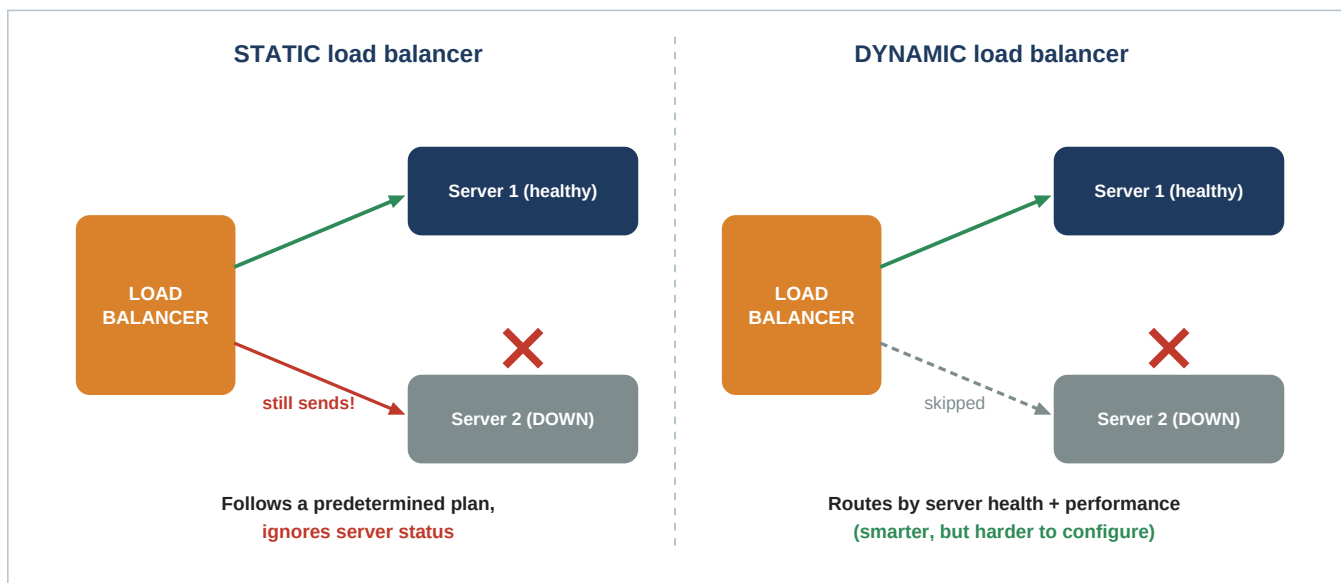


Figure 10: Static follows a fixed plan; dynamic reacts to server health and performance

- **Static:** routes traffic by a **predetermined plan** and **ignores server status**. Example: 'send every 1st request to A, 2nd to B' even if B is dead.
- **Dynamic:** routes by **server health and performance**. Example: it notices Server 2 is down and sends nothing there; it prefers the server with less load. **Downside:** it is **harder to configure**.

5.5 Layer 4 vs Layer 7 load balancers

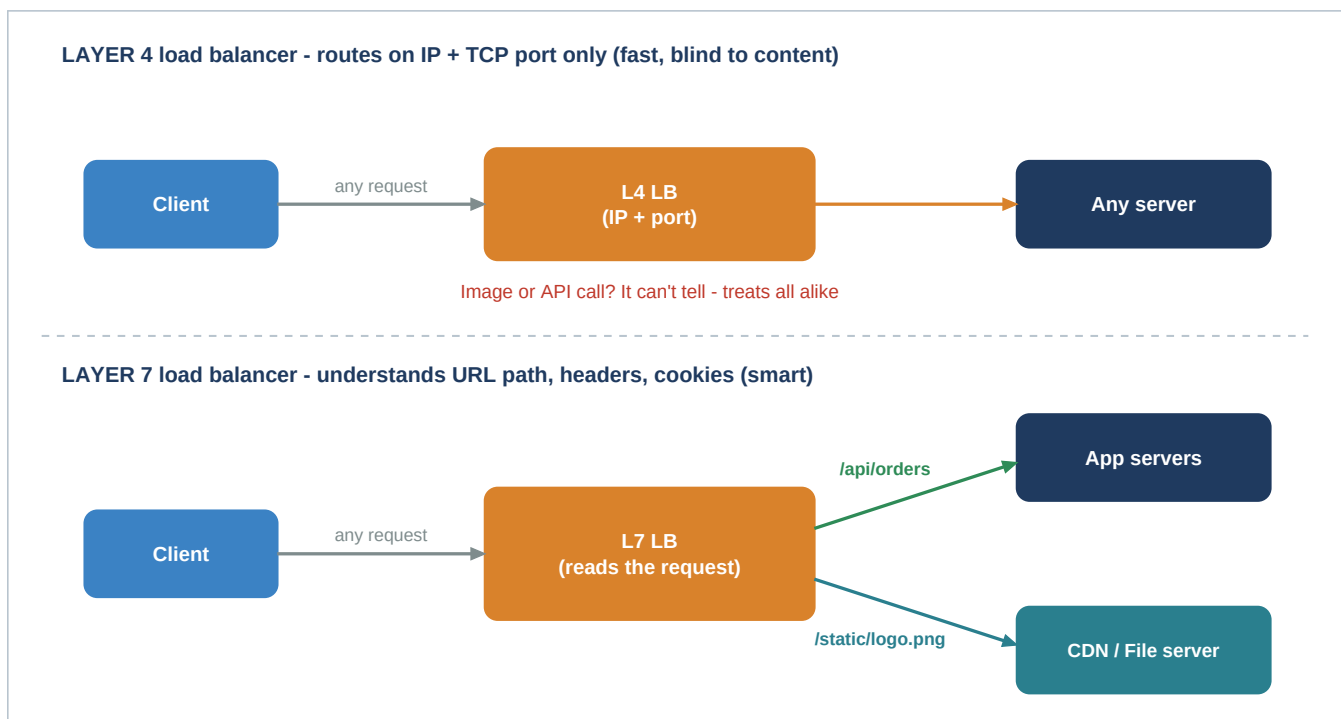


Figure 11: Layer 4 routes on IP/port; Layer 7 understands the request content

Aspect	Layer 4 (Transport layer)	Layer 7 (Application layer)
Looks at	IP address + TCP port only	Full request: URL path, headers, cookies

Aspect	Layer 4 (Transport layer)	Layer 7 (Application layer)
Content of packet matters?	No	Yes, used for the routing decision
Speed	Faster (no need to read content)	Slower (must intercept and read content)
Smartness	Basic	Content-based routing
Example	Spread TCP connections across 3 servers	/api -> app servers, /images -> file server

5.6 Load balancing algorithms

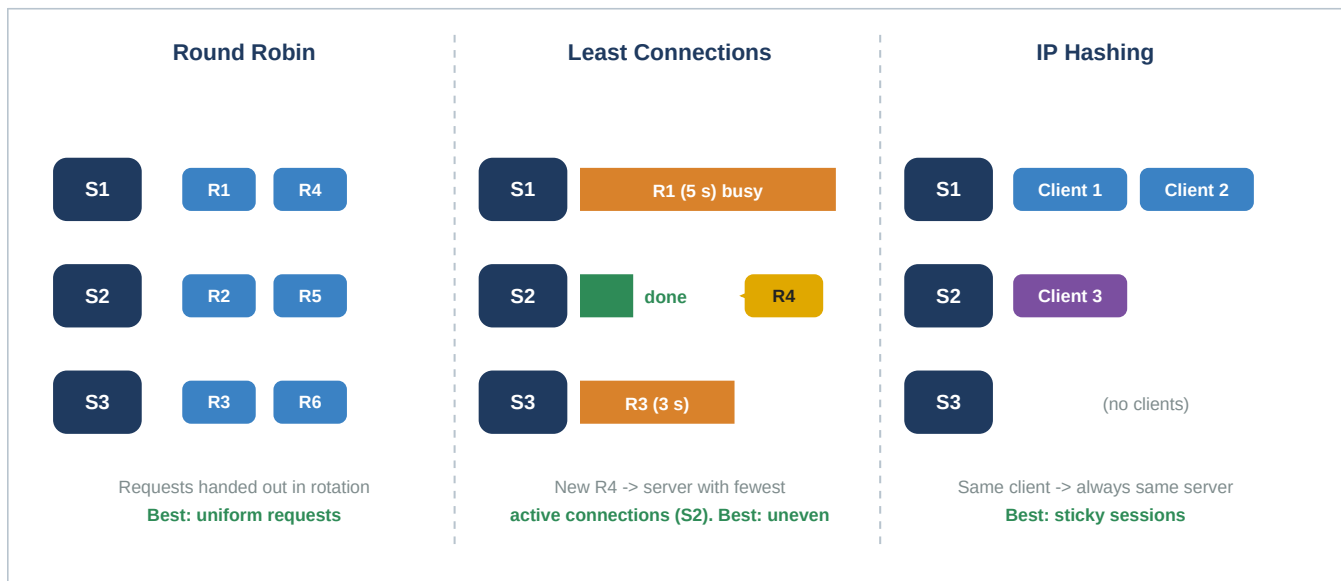


Figure 12: Round robin, least connections and IP hashing (the idea behind sticky sessions)

(a) Round Robin

Distributes requests **evenly, one by one, in rotation**.

EXAMPLE

Requests R1, R2, R3, R4 and servers A, B, C: R1 -> A, R2 -> B, R3 -> C, R4 -> back to A. Simple and fair when all requests cost about the same.

(b) Least Connections

Chooses the server with the **fewest active requests** right now.

EXAMPLE

R1 -> A (takes long), R2 -> B (finishes quickly), R3 -> C (takes long). When R4 arrives, B has 0 active connections while A and C still have 1 each, so R4 goes to B. Great when some requests are heavy and some are light.

(c) Sticky sessions

Makes sure **one user stays on the same server for the entire session**. The first request is routed (say to Server 1) and every later request in that session also goes to Server 1, never Server 2.

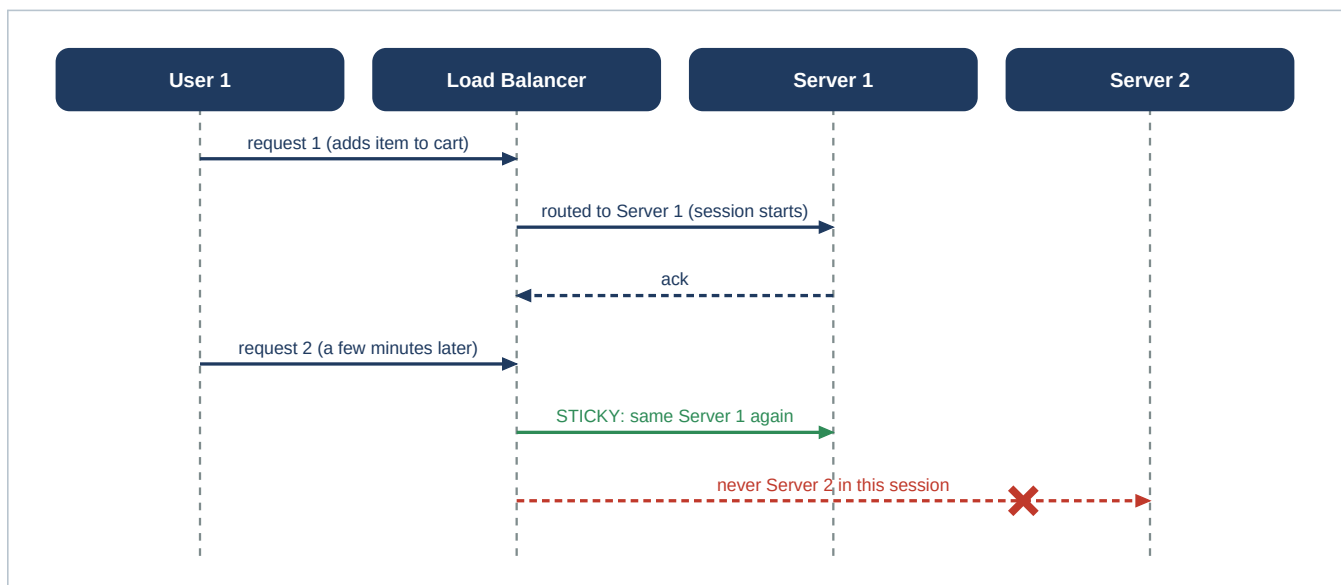


Figure 13: Sticky session: the same user keeps returning to the same server

EXAMPLE

Shopping cart: you add an item while on Server 1, which remembers it. If your next request went to Server 2 it would not know about your cart. With sticky sessions every request lands on Server 1, giving a **consistent experience**. Useful for shopping carts, dashboards and e-commerce sites.

6. Caching

6.1 What is caching?

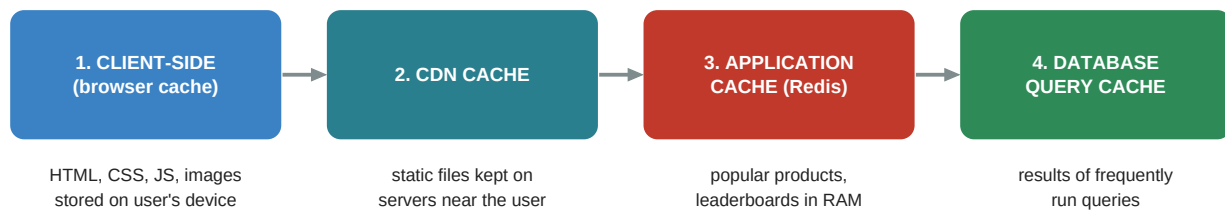
Caching is storing **frequently accessed data in a faster storage (cache memory)** so that future requests are served faster. The expensive work (calculating or fetching) is done once; later requests reuse the stored result with **no recalculation or refetching**.

EXAMPLE

Your teacher repeatedly gets asked 'what is the exam date?'. Instead of checking the master schedule each time, she writes it on the board. Everyone gets the answer instantly. The board is the cache.

6.2 Types of caching

Four places where caching happens on the path of a request



Each layer can answer early (CACHE HIT) so the request never reaches the next layer

Example: iPhone page opened twice -> 2nd time images load from your own browser cache, the specs come from Redis, the DB is never touched

Figure 14: Caches exist at several layers between the user and the database

- **Client-side caching (browser cache):** the browser stores page resources (HTML, CSS, JS, images) on the user's computer. Next visit the page loads faster and the server gets less load. Example: the Twitter logo and stylesheet do not download again on your second visit.
- **CDN caching:** static files (HTML, CSS, JavaScript) are cached on servers near the user to cut latency (see chapter 7).
- **Application-level caching:** backend servers use in-memory caches such as **Redis or Memcached** to store hot data like popular products or leaderboards. Example: the top-10 game leaderboard is computed once and read from Redis thousands of times.
- **Database query caching:** frequently executed queries and their results are cached. Example: during an iPhone sale, everybody searches 'iPhone'. The result of that query is cached so the DB does not recompute it for every shopper.

6.3 Cache hit and cache miss

- **Cache hit:** the data is found in the cache, so it is served **fast**.
- **Cache miss:** the data is not in the cache, so the system must fetch or recalculate it from the main store (**slower**) and usually saves it in the cache for next time.

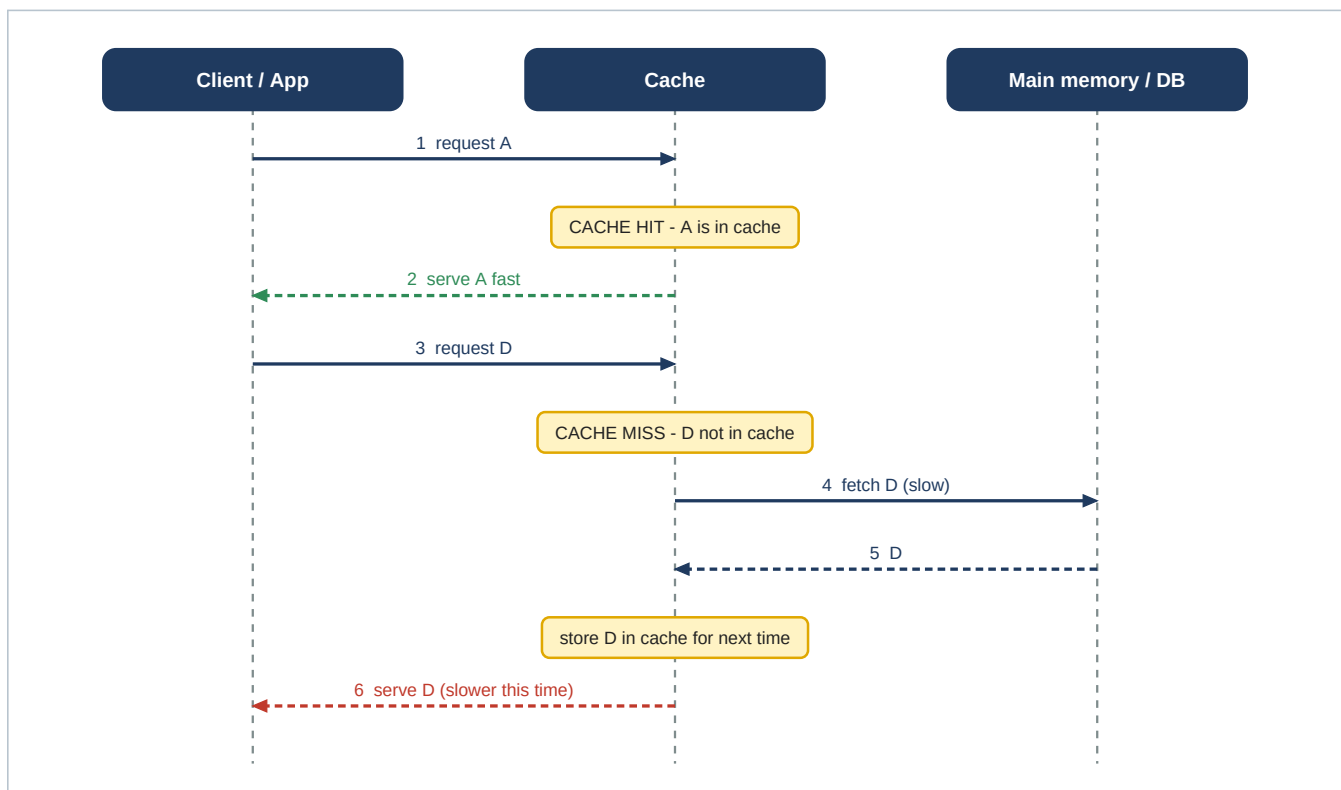


Figure 15: Cache hit (A) vs cache miss (D)

6.4 Cache eviction algorithms (what to remove when the cache is full)

Cache memory is small and costly, so when it is full an item must be **evicted** to make room. The algorithm decides which one.

(a) LRU - Least Recently Used

Evict the item that has **not been used for the longest time**. Cache size = 3, access order **A, B, C, A, D, E**:

LRU | cache size = 3 | access order: A B C A D E

Request	A	B	C	A	D	E
Slot 1	A	A	A	A	A	A
Slot 2	-	B	B	B	D	D
Slot 3	-	-	C	C	C	E
Result	MISS	MISS	MISS	HIT	MISS	MISS
Evicted	-	-	-	-	B	C

D arrives: A and C were used more recently than B -> evict B (least recently used).

E arrives: C is now the stalest (A and D are fresher) -> evict C. Final cache: A, D, E

gold cell = item just placed in cache

Figure 16: LRU step-by-step trace

- A, B, C are misses and fill the cache. The second A is a **hit** (A is now recently used).
- D arrives, cache full. Recency: B is the oldest used, so **B is evicted** (D takes B's slot).
- E arrives. A and D are fresh, C is now the stalest, so **C is evicted**. Final cache: A, D, E.

(b) FIFO - First In, First Out

Evict the item that **entered the cache first**, regardless of how often or recently it was used.

FIFO cache size = 3 access order: A B C A D E						
Request	A	B	C	A	D	E
Slot 1	A	A	A	A	D	D
Slot 2	-	B	B	B	B	E
Slot 3	-	-	C	C	C	C
Result	MISS	MISS	MISS	HIT	MISS	MISS
Evicted	-	-	-	-	A	B

D arrives: A entered first -> evict A, even though A was just used (a HIT)!

E arrives: B entered next -> evict B. Final cache: D, E, C (age order ignored for usage)

gold cell = item just placed in cache

Figure 17: FIFO step-by-step trace with the same access order

EXAMPLE

Same sequence A, B, C, A, D, E: when D arrives, **A** is evicted because it came first, even though A was just used. That is FIFO's weakness. Then E evicts B (the next oldest).

(c) LFU - Least Frequently Used

Evict the item with the **lowest access count**.

LFU | cache holds A, B, C | new item E arrives

Item A  accessed 5 times

Item B  accessed 3 times

Item C  accessed 1 time

EVICT C

ADD E

Least frequently used = lowest access count. A and B are likely to be requested again (5 and 3 times) so they stay; C (only 1 time) goes. Final cache: A, B, E

Figure 18: LFU: C has the lowest frequency, so it is evicted for E

EXAMPLE

A was accessed 5 times, B 3 times, C once. When E arrives, C is removed because it is the least likely to be requested again; A and B (popular) stay.

Algorithm	Evicts	Good when	Weakness
LRU	Not used for the longest time	Recent use predicts future use	Needs tracking of use order
FIFO	Oldest entry	Simplicity matters	May evict a very popular item
LFU	Lowest access count	Some items are consistently popular	Old once-popular items can linger

6.5 When to use caching and when to avoid it

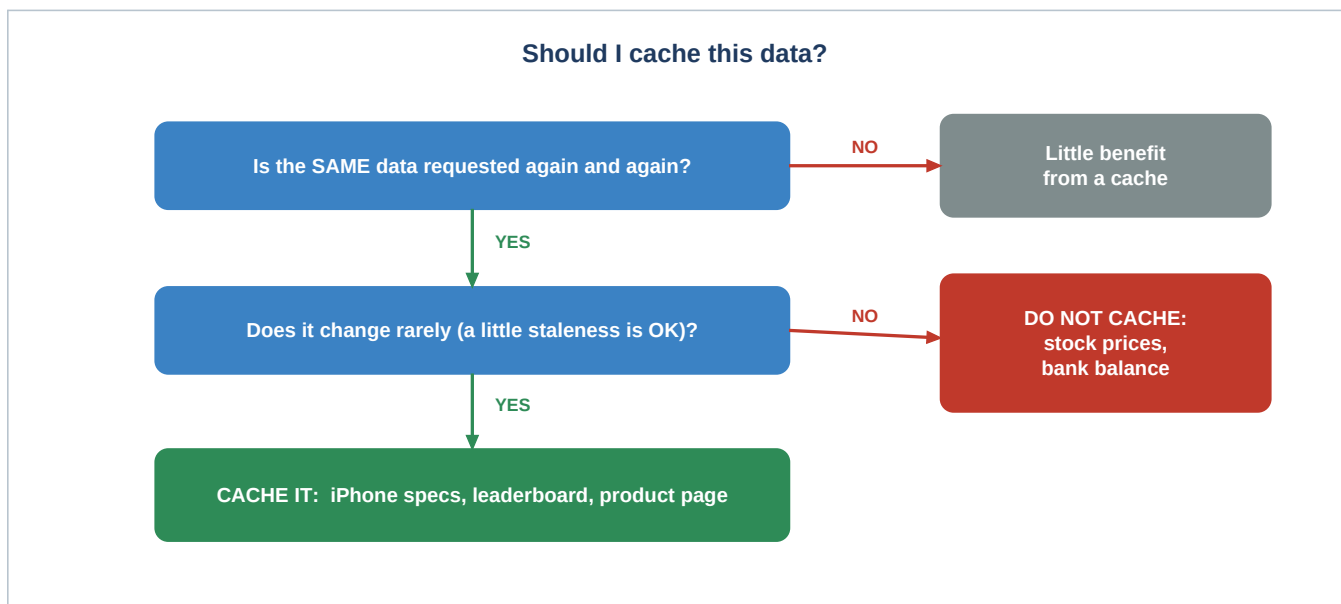


Figure 19: Decision flow for caching

Use caching when the same data is requested again and again, the data does not change often, and you want ultra-fast performance.

EXAMPLE

Product details of the iPhone on sale, a city's weather summary refreshed every 10 minutes, a game leaderboard.

Avoid caching when data changes frequently and you must always show **fresh** data. A little slowness is acceptable; stale data is not.

EXAMPLES

Stock market prices: showing a 5-minute-old price could cause wrong trades.

Bank balance: it must never show Rs 1,00,000 after you spent Rs 90,000 just because the cache has not updated. The query may be slower but the data must be correct.

7. CDN (Content Delivery Network)

7.1 What is a CDN?

A CDN is a **network of servers located around the world** that **cache and deliver static content from the server closest to the user**, avoiding the latency of a long trip to the main server.

7.2 Analogy: warehouses

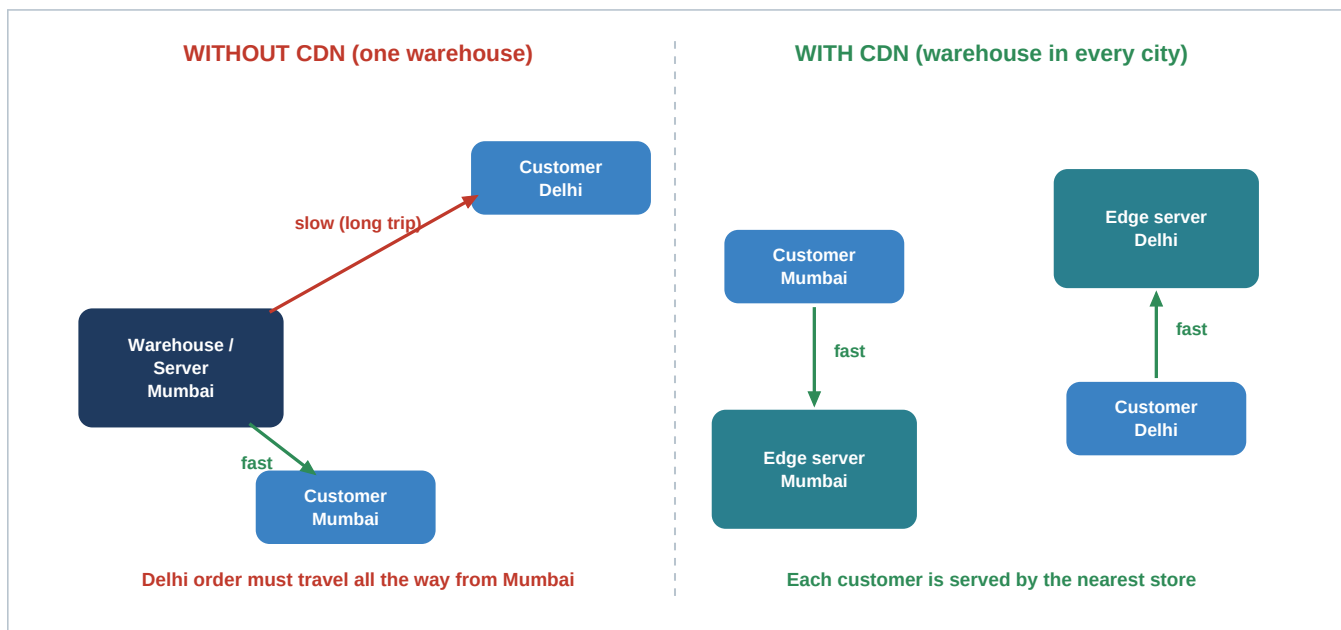


Figure 20: Without a CDN one warehouse serves everyone; with a CDN every city has its own

EXAMPLE (ANALOGY)

Without CDN: an e-commerce company has a single warehouse in Mumbai. A customer in Mumbai gets the parcel quickly, but a customer in Delhi waits a long time for it to travel.

With CDN: there are warehouses in Mumbai *and* Delhi. Each customer is served from the nearest one, so deliveries are fast for everyone. The 'warehouses' are CDN edge servers; the 'parcels' are your images, videos and scripts.

7.3 How a CDN works: the three components

- **Origin server:** holds the **original, authoritative version** of the content (source of truth). Any change (new design, new file) is made here first and later reflected on the edge servers.
- **Edge servers (Points of Presence, PoPs):** servers in many geographic locations that keep **cached copies** of origin content and deliver it from nearby.
- **DNS server:** when a client asks for a site, DNS responds with the address of the **nearest edge server** instead of the origin so content can be served faster.

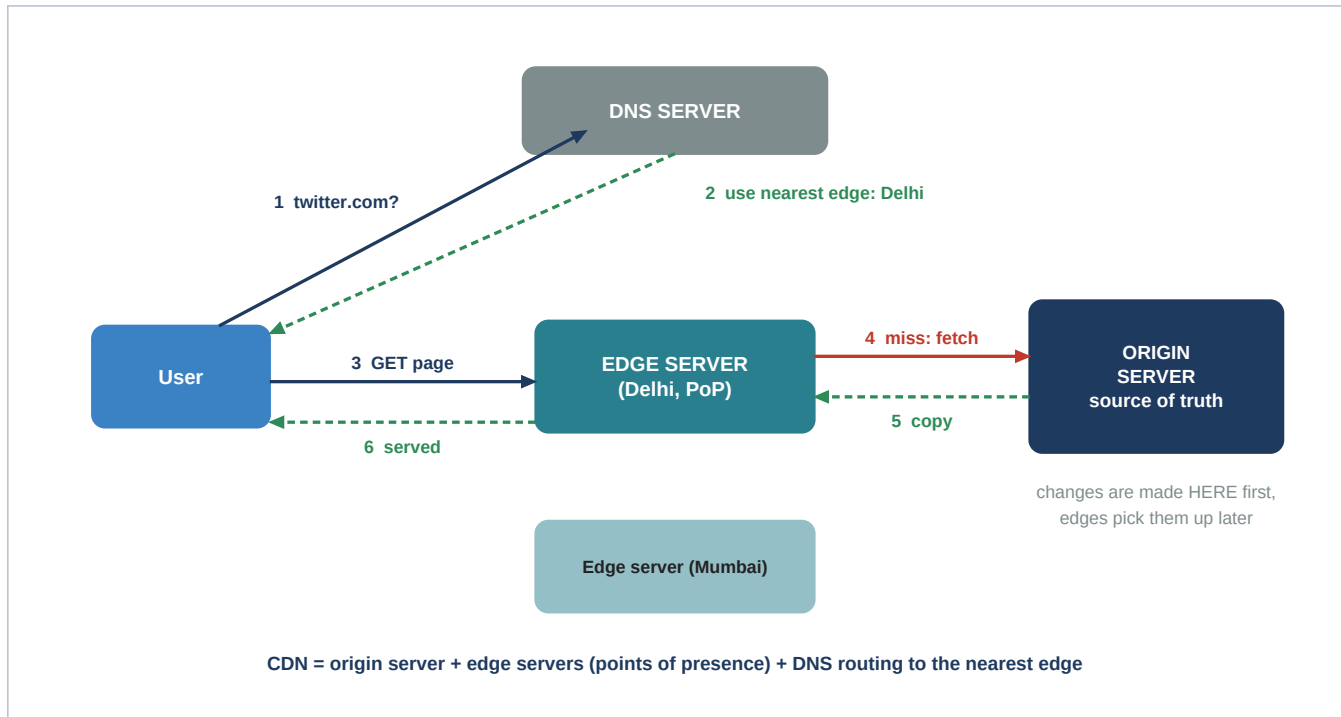


Figure 21: Request flow through DNS, the nearest edge server and, on a miss, the origin

EXAMPLE

A user in Delhi opens `twitter.com`. DNS says 'use the Delhi edge server'. If the Delhi edge already has the logo and CSS (cache hit), it serves them instantly. If not (cache miss), it fetches them from the origin, keeps a copy and serves the user. The next Delhi user gets a hit.

7.4 What does a CDN cache?

Mostly **static assets** that rarely change: **HTML, CSS, JavaScript, images, videos, PDFs, fonts and icons**.

EXAMPLE

A phone-review site offers a PDF of the iPhone 15 specifications. Once generated, it rarely changes, so it is safe to cache on edge servers. Likewise a website's icon or font changes maybe once in many years, so it is ideal for CDN caching.

Not suitable: data that is personal or changes per request, such as your account balance or a live API response.

8. SQL vs NoSQL Databases

Every real application has a database. Choosing the **wrong type** (SQL when NoSQL is required, or the reverse) hurts the **efficiency of data retrieval, storage and processing** and therefore the whole system.

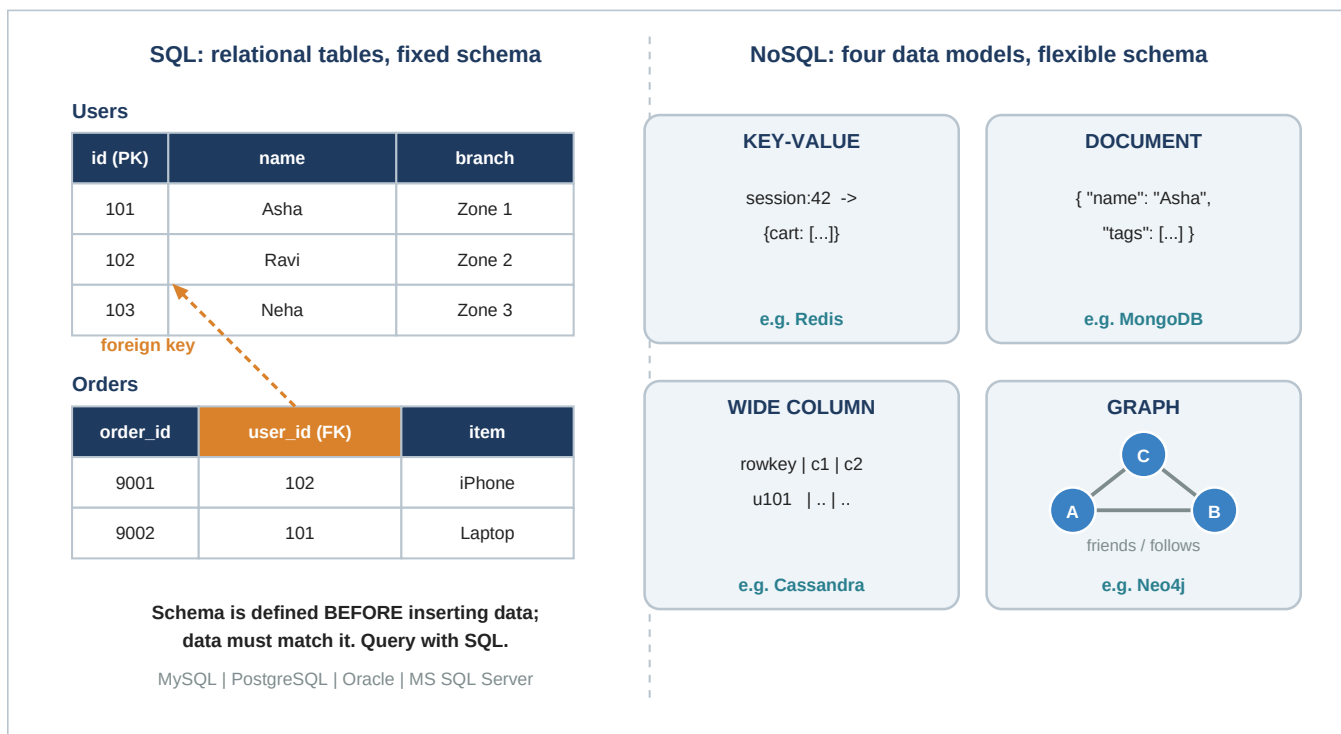


Figure 22: SQL (tables with fixed schema and relations) vs the four NoSQL data models

8.1 SQL databases (relational)

- **SQL = Structured Query Language**, used to query data in a **relational** database.
- Data is stored in **tables of rows and columns** with a **predefined schema** defined *before* any data is inserted.
- Tables are related using **foreign keys**.
- Schemas are **fixed and rarely changed**; every insert must match the schema and formatting rules.
- Data is read and updated using **SQL queries**.
- Examples: **MySQL, Microsoft SQL Server, Oracle, PostgreSQL**.

EXAMPLE

A **Users** table (id, name, branch) and an **Orders** table (order_id, user_id, item). user_id in Orders is a foreign key pointing to Users.id, so you can query 'all orders placed by Ravi' with a JOIN. Trying to insert an order with a text value in the numeric user_id column is rejected.

8.2 NoSQL databases (non-relational)

'Not only SQL'. There is **no predefined schema** (often **schemaless**) and they support several data models. They are generally **easy to scale horizontally**.

Model	How data looks	Example DB	Example use
Key-value	key -> value pairs	Redis	Sessions, caching, shopping cart
Document	JSON-like documents	MongoDB	Product catalogs, user profiles
Wide column	Rows with flexible columns across many nodes	Cassandra	Huge time-series / activity logs

Model	How data looks	Example DB	Example use
Graph	Nodes and relationships	Neo4j	Social networks, friend suggestions

EXAMPLE

A social network stores 'who follows whom' in a **graph** DB: finding 'friends of friends' is a quick traversal. A **document** DB stores each product as a flexible JSON document so a phone and a T-shirt can have different fields with no schema change.

8.3 When to use which

Use SQL when...	Use NoSQL when...
Queries must follow ACID (for example transactions)	Schema is flexible or you are just getting started
You have complex queries or reporting	You face massive traffic and need low read latency
Data has complex relationships	You need high-speed caching or session data
Any transaction-related work (bank, payments)	Social-network style data (graph DBs)

KEY TAKEAWAY

For anything involving **transactions** (money, orders, inventory) choose **SQL**, because it guarantees consistency and ACID.

8.4 ACID properties

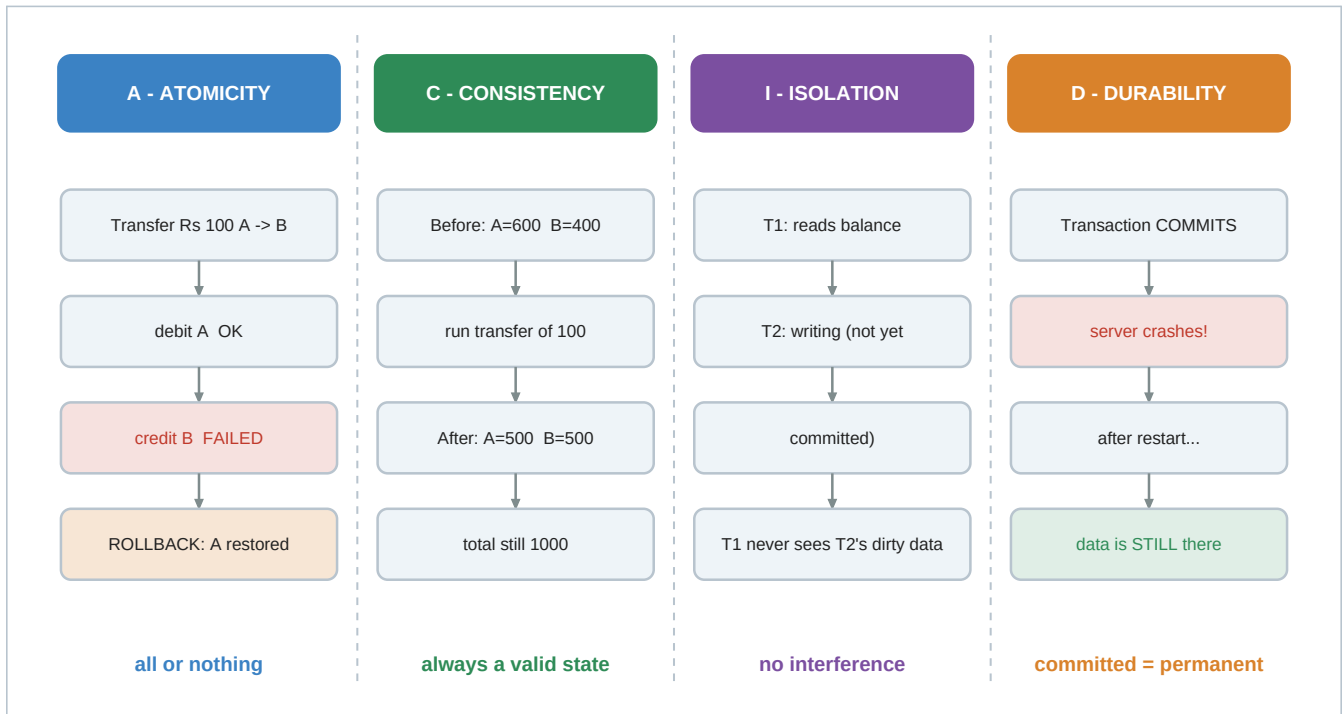


Figure 23: The four ACID guarantees illustrated

- A - Atomicity:** a transaction is **all or nothing**; no partial updates. Example: transferring Rs 100 from A to B debits A *and* credits B. If the credit fails, the debit is rolled back.

- **C - Consistency:** the database always moves from one **valid state to another**. Example: total money A+B stays 1000 before and after the transfer; rules like 'balance cannot be negative' are never broken.
- **I - Isolation:** concurrent transactions **do not interfere**: no dirty reads, no lost updates, no race conditions. Example: while T2 is still updating a balance, T1 cannot read its half-finished value.
- **D - Durability:** once a transaction is **committed** it is **permanent**, even if the server crashes right after. Example: you see 'Payment successful', the power goes out, and after restart the payment is still recorded.

9. Vertical vs Horizontal Scaling

Scaling is the process of **increasing a system's capacity** to handle more traffic, more users, more data or sudden traffic spikes. There are two strategies.

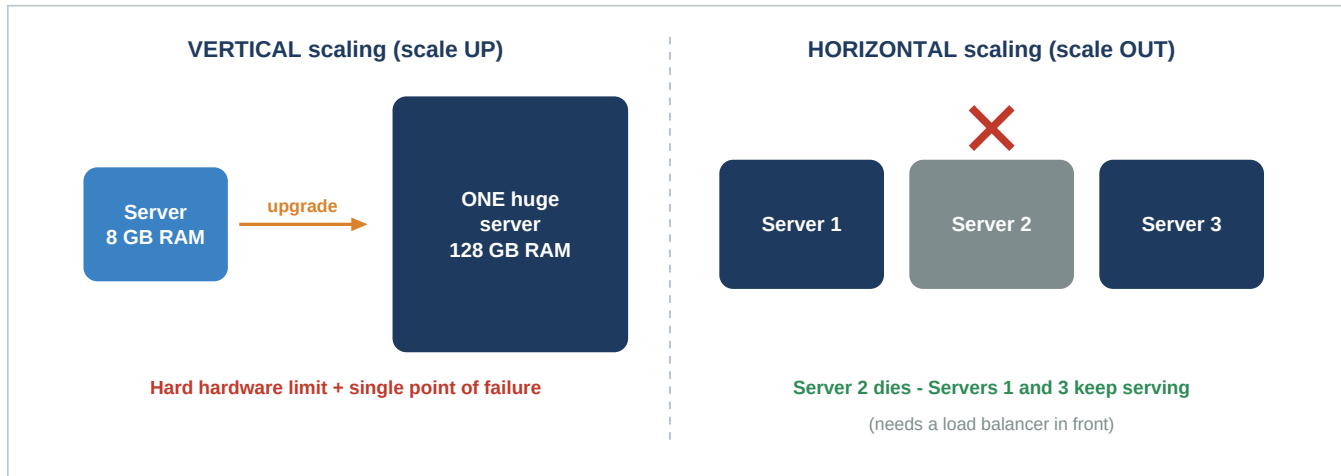


Figure 24: Vertical (scale up) vs horizontal (scale out)

9.1 Vertical scaling (scale up)

Make the **single server more powerful**: add more CPU, RAM, disk or network bandwidth. You **upgrade the same machine**, making the smaller box bigger.

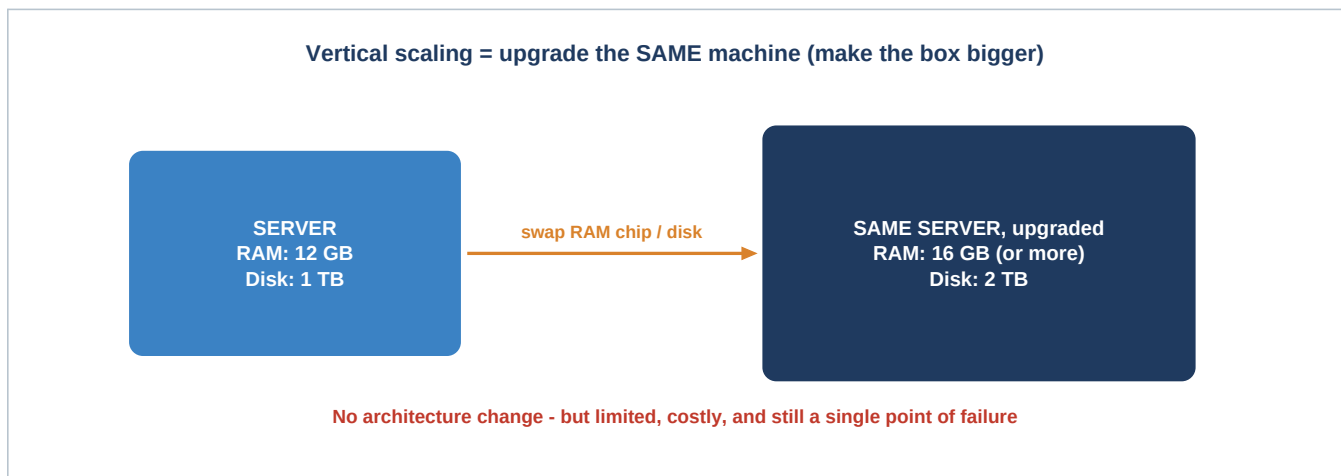


Figure 25: Upgrading the same machine

EXAMPLE

A server has 12 GB RAM and a 1 TB disk. As traffic grows you upgrade it to 16 GB (or more) RAM and a 2 TB disk. It is like pulling out the old RAM chip and plugging in a bigger one.

Pros

- **Simple and easy to manage:** no architecture change, only hardware.
- **Fast reads and writes:** one server means no network hops between machines.

Cons

- **Hard limit:** you cannot upgrade a server infinitely.
- **Costly:** bigger configurations get disproportionately expensive. Buying several 16 GB machines is cheaper than one very large machine.
- **Single point of failure:** if this one box goes down, the whole application is down; there is no backup.

9.2 Horizontal scaling (scale out)

Instead of upgrading one machine, **add more machines (nodes)**. Each handles a portion of the traffic or data. If one server is down the others keep serving.

EXAMPLE

Instead of upgrading your app server, you add three more behind a load balancer. Requests are distributed across all four; one can fail and nobody notices. Some servers may even be in locations closer to users. (With vertical scaling there is only one server, so all requests from India and the USA go to the same machine, perhaps in Europe.)

Pros

- **Highly scalable:** keep adding machines.
- **Cost-effective:** several normal machines cost less than one extreme machine.
- **Fault tolerant:** no single point of failure and no single overloaded server.

Cons

- **Complex:** needs a load balancer and an architecture designed for multiple servers; moving from a single-server design later may require a **complete architecture change**.
- **Data consistency is hard:** all servers must hold correct, current data. A stale server could show old data, unacceptable for banking or e-commerce.
- **Cross-node communication** must be reliable and is hard to implement.

Horizontal scaling challenge: data consistency across machines



Every update must be synced over the network. A user hitting Server 3 sees old data - unacceptable for banking or e-commerce.

Also: complex architecture, cross-node communication, hard to debug

Figure 26: The data-consistency challenge of horizontal scaling

Aspect	Vertical	Horizontal
Method	Bigger machine	More machines
Limit	Hardware ceiling	Practically unlimited
Cost at large scale	Very high	Lower per unit of capacity
Single point of failure	Yes	No
Complexity	Low	High
Data consistency	Easy (one machine)	Harder (many copies)
Architecture change	None	Required

KEY TAKEAWAY

Start vertical (simple). When limits, cost or availability become a problem, go horizontal with a load balancer.

10. Database Sharding

10.1 What is sharding?

Sharding is a database architecture pattern where a **large database is split into smaller, faster, more manageable databases called shards**. Each shard is an **independent database instance** that holds a **subset of the total data**.

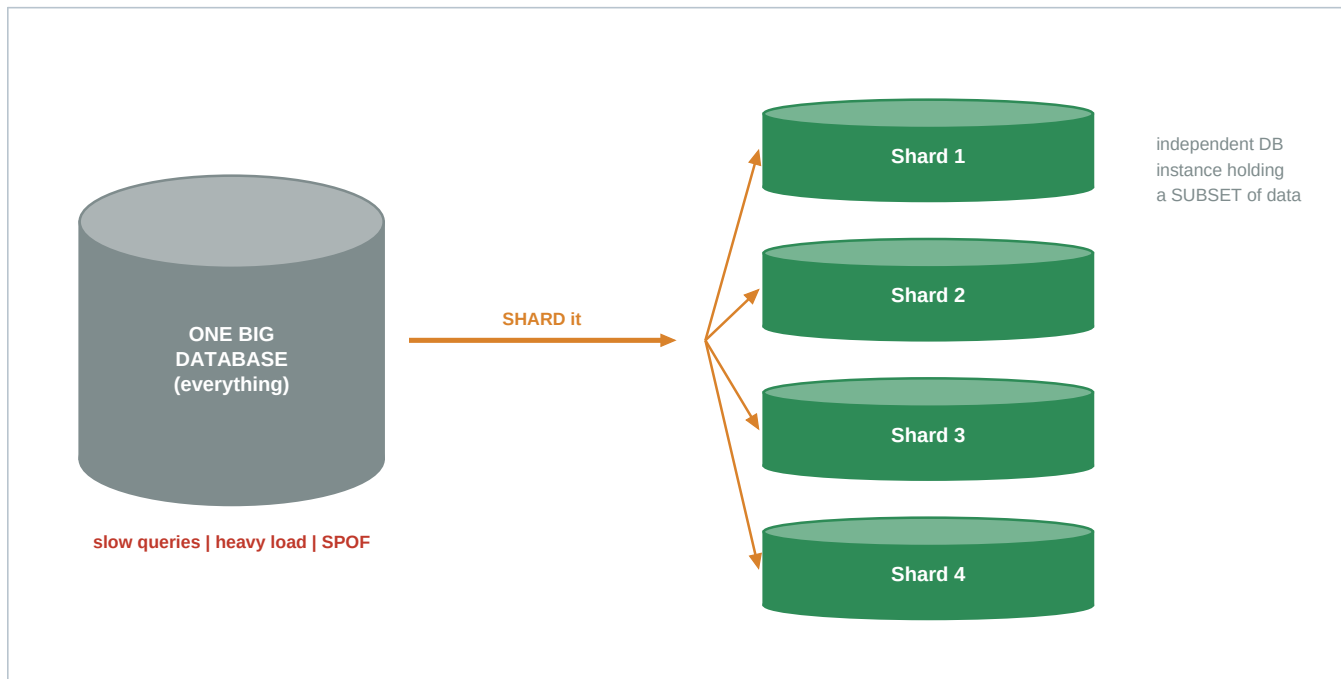


Figure 27: One big database split into independent shards

10.2 Why shard?

- **Huge tables slow queries:** a platform like Facebook may have billions of rows; scanning them is slow.
- **A single database cannot be scaled vertically forever**, and scaling it horizontally is hard (see chapter 9).
- **High read and write load** lands on one database.

- **Single point of failure:** one database means one thing to lose.

EXAMPLE

A social app's users table has 500 million rows on one server. Every login query is slow and the server is maxed out. After sharding into 4 shards, each holds about 125 million rows and handles about a quarter of the traffic.

10.3 How sharding works: two decisions

1) Choose a **shard key**. 2) Apply a **sharding algorithm**.

Step 1: Choose the shard key

The shard key is a **column (field) of your data used to decide which shard stores a record**. It should be **unique and stable** so that a record always maps to the same shard.

Choosing the shard key (social media example)

user_id	name	location	profile photo
101	Asha	Pune	p1.jpg
102	Ravi	Delhi	p2.jpg
103	Asha	Goa	p3.jpg

GOOD KEY: unique,
never changes

BAD keys: names repeat (Asha twice), location and photo can change

A record must always map to the **SAME** shard, so the key must be stable and unique

Figure 28: user_id is a good shard key; names and locations are not

EXAMPLE

On a social media site you can change your name, location and photo, and two users can share a name. But your **user ID** is unique and never changes, so it is the ideal shard key.

Step 2: Apply a sharding algorithm

(a) Hash-based sharding

Apply a **hash function** to the shard key and take the result modulo the number of shards. The result is the shard number.

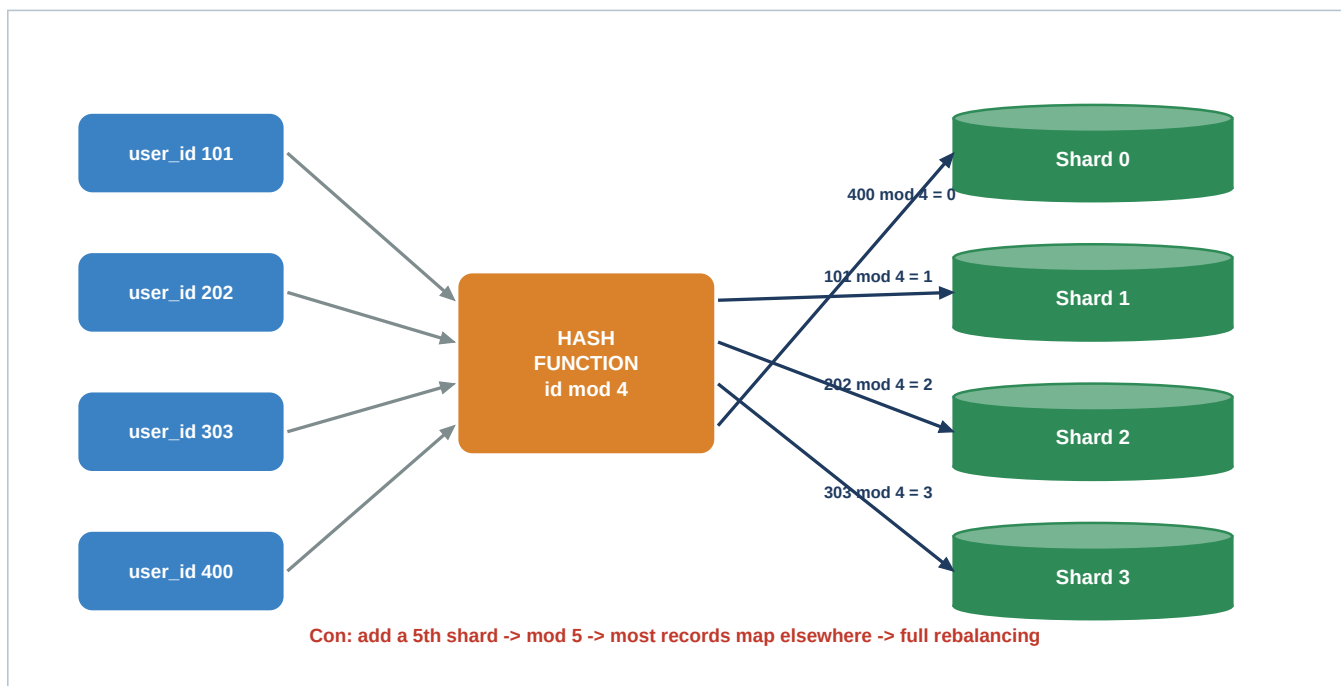


Figure 29: Hash-based sharding with 4 shards (id mod 4)

EXAMPLE

4 shards: user 101 -> $101 \bmod 4 = 1$ -> Shard 1; user 202 -> $202 \bmod 4 = 2$ -> Shard 2; user 303 -> $303 \bmod 4 = 3$ -> Shard 3; user 400 -> $400 \bmod 4 = 0$ -> Shard 0. Data spreads evenly.

Pro: even distribution, simple. **Con:** hard to add or remove shards. If you go from 4 to 5 shards the formula changes to mod 5 and most records now map to a different shard, so you must **rebalance (move) a large amount of data**.



Figure 30: Why changing the number of shards moves most data (consistent hashing reduces this)

(b) Range-based sharding

Split the key space into **ranges** and assign each range to a shard.

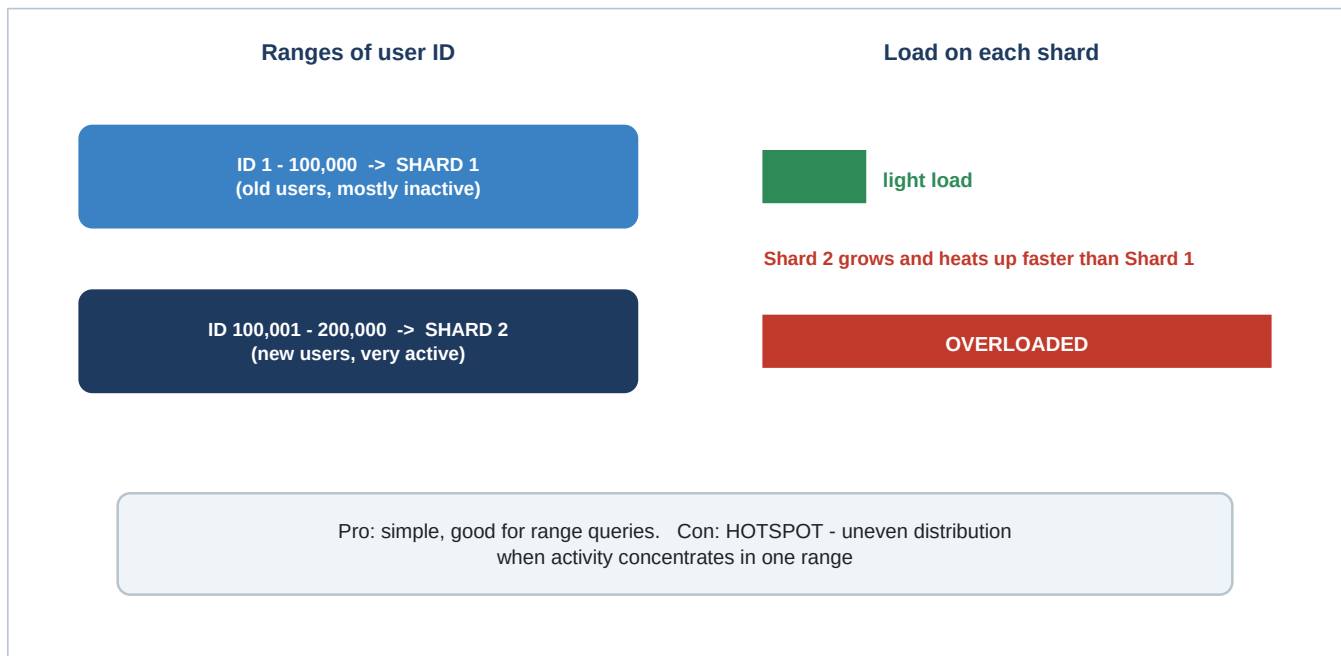


Figure 31: Range-based sharding and its hotspot problem

EXAMPLE

User IDs 1 - 100,000 go to Shard 1; 100,001 - 200,000 go to Shard 2. Simple to understand, and range queries ('users 5,000 to 6,000') hit one shard.

Problem (hotspot): old users in Shard 1 are mostly inactive, while recently joined users in Shard 2 are very active. Shard 2 becomes overloaded and grows faster than Shard 1: **uneven distribution**.

(c) Directory-based sharding

Maintain a **lookup table (directory)** that maps each shard-key value to a specific shard.

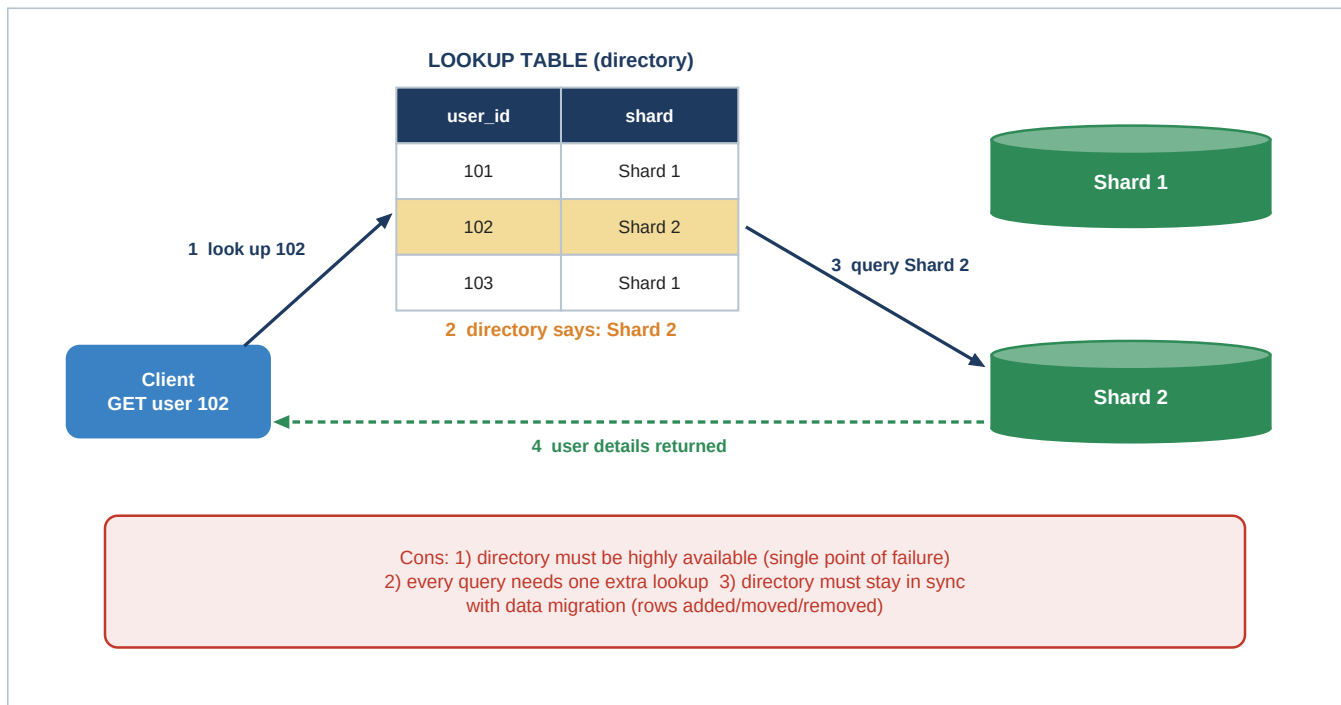


Figure 32: Directory-based sharding: look up the shard, then query it

EXAMPLE

Lookup table: user 101 -> Shard 1, 102 -> Shard 2, 103 -> Shard 1. Request 'get user 102': the system consults the directory (answer: Shard 2), queries Shard 2 and returns the details.

Pro: very flexible (you can place any record anywhere and move it by editing the table). **Cons:**

- The directory **must be highly available** because it is a single point of failure.
- **Every query needs one extra lookup** (directory first, then the shard).
- The directory **must stay in sync** with data migration: when records are added, moved or removed, the lookup table must be updated too.

Algorithm	How it decides	Pros	Cons
Hash-based	hash(key) mod number of shards	Even spread, simple	Changing shard count forces big rebalancing
Range-based	Key falls in a defined range	Simple, good for range queries	Hotspots / uneven load
Directory-based	Lookup table key -> shard	Flexible placement	Directory is a SPOF, extra lookup, must sync

11. Big Picture, Cheat Sheet and Interview Q&A

11.1 How the building blocks fit together

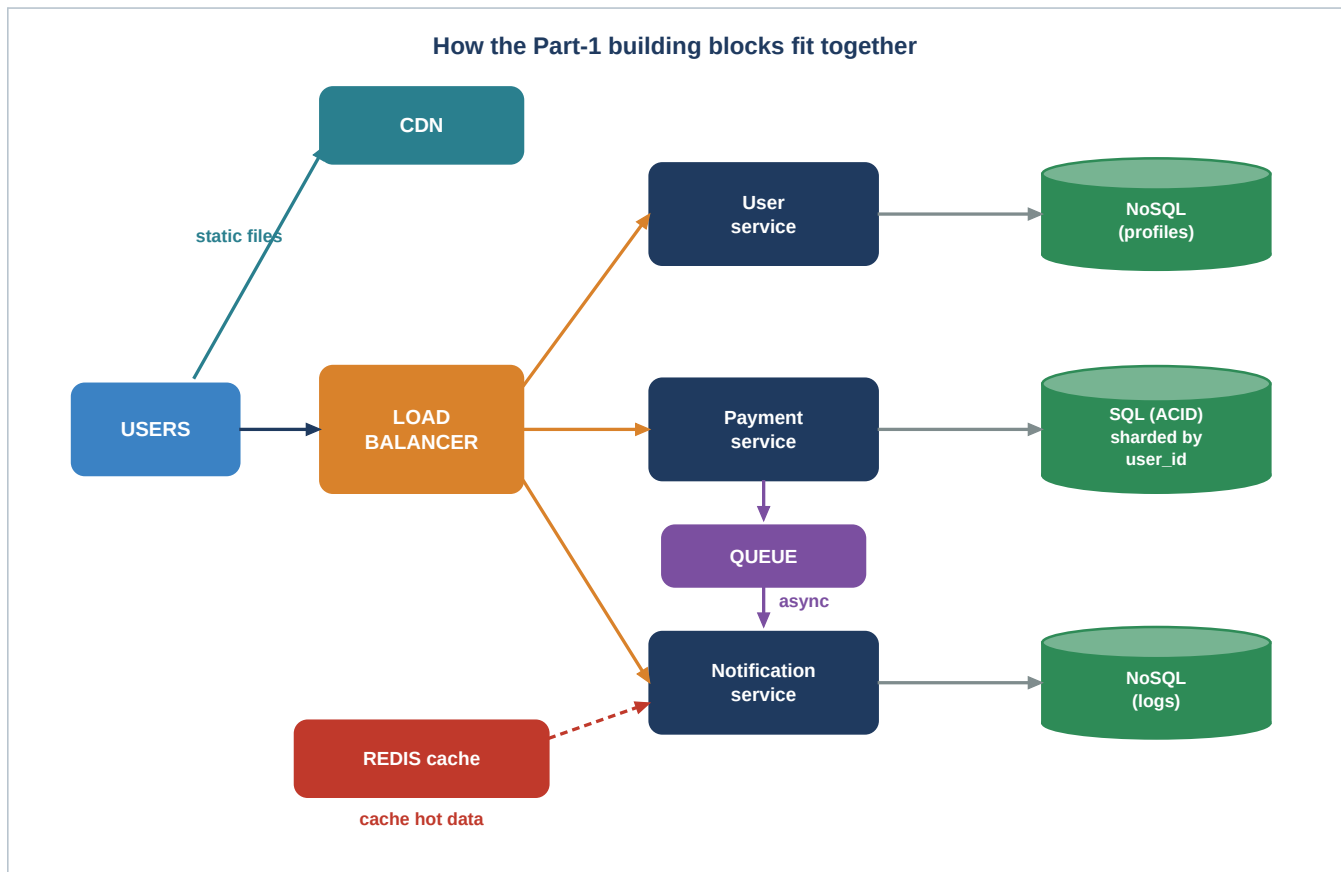


Figure 33: A typical architecture assembled from Part-1 concepts

Users fetch **static files from a CDN**. Dynamic requests go to a **load balancer** that spreads them across **microservices**. Each service has its own database, chosen by need: **SQL with ACID** (sharded by user_id when large) for payments, **NoSQL** for profiles and logs. Slow work such as notifications happens **asynchronously via a queue**. A **Redis cache** keeps hot data in memory.

11.2 One-line summary per chapter

Concept	Remember this
System design	Plan so the system survives scale (many users) and latency (distance).
Client-server	Client never touches the DB; the server is the secure middleman. 3-tier is the standard.
Monolith vs microservices	Monolith = easy start, one failure hurts all. Microservices = independent, resilient, complex.
Sync vs async	Sync waits (blocking). Async drops into a queue and moves on.
Load balancer	Spreads traffic, hides failures. Round robin / least connections / sticky; L4 fast, L7 smart.
Caching	Store hot data in fast memory. Know hit/miss and LRU/FIFO/LFU. Do not cache must-be-fresh data.
CDN	Origin + edge servers + DNS; serves static content from the nearest location.
SQL vs NoSQL	SQL: fixed schema, relations, ACID, transactions. NoSQL: flexible, scalable, 4 models.
ACID	Atomicity, Consistency, Isolation, Durability.
Scaling	Vertical = bigger machine (limit, SPOF). Horizontal = more machines (complex, consistent data needed).
Sharding	Split a huge DB. Pick a unique, stable shard key; hash, range or directory algorithm.

11.3 Interview Q&A

Q: Why is system design important?

A: To ensure software keeps working in the real world despite large user growth and geographic latency.

Q: Why can't a client talk to the database directly?

A: It is insecure. The server mediates, authenticating and validating before running queries.

Q: Two-tier vs three-tier?

A: Two-tier: client + server (small apps). Three-tier: client, application server, database (scalable, maintainable).

Q: When choose a monolith over microservices?

A: When starting out: simpler to build, test and fast with no network calls. Move to microservices as you scale.

Q: Sync vs async communication?

A: Sync blocks until a response. Async uses a queue; the sender moves on and gets a completion event later.

Q: What does a load balancer give you?

A: High availability, traffic optimization, easy horizontal scaling and spike handling.

Q: Static vs dynamic load balancing?

A: Static follows a fixed plan ignoring server state; dynamic uses health and performance but is harder to configure.

Q: Layer 4 vs Layer 7?

A: L4 routes by IP/port (fast, content-blind); L7 routes by URL/headers/cookies (smarter, slower).

Q: When use sticky sessions?

A: Shopping carts, dashboards, anything where the server holds per-user session state.

Q: What are LRU, FIFO and LFU?

A: Evict least recently used / oldest inserted / least frequently used item when the cache is full.

Q: When should you not cache?

A: Frequently changing data that must be fresh: stock prices, bank balances.

Q: Components of a CDN?

A: Origin server (truth), edge servers/PoPs (cached copies), DNS (routes to nearest edge).

Q: SQL or NoSQL for payments?

A: SQL, because transactions need ACID guarantees.

Q: Explain ACID with an example.

A: Money transfer: debit and credit both happen or neither (A); totals stay valid (C); concurrent transfers do not interfere (I); committed transfer survives a crash (D).

Q: Vertical vs horizontal scaling?

A: Vertical upgrades one machine (simple but limited and a SPOF). Horizontal adds machines (scalable, fault tolerant, complex).

Q: What is a shard key and what makes it good?

A: The field deciding a record's shard; it should be unique and never change (e.g. `user_id`).

Q: Hash vs range vs directory sharding?

A: Hash: even but painful to resize. Range: simple but hotspots. Directory: flexible but the lookup table is a SPOF and extra hop.

11.4 Notes on the source material

- The transcript is auto-generated, so a few terms were garbled: 'STP/STPS' means HTTP/HTTPS, 'radius' means Redis, 'mem cache' means Memcached. These notes use the correct terms.
- Where the speaker misspoke, the notes use the correct statement: the sentence 'in monolithic architecture we have many services' refers to **microservices**; the example describing a hardware upgrade as 'horizontal scaling' is **vertical** scaling; and 'SQL databases are easy to scale horizontally' refers to **NoSQL** databases.
- The LRU example lists the access order as A B C A B D E but the walkthrough skips the second B. These notes use the walkthrough's order (A, B, C, A, D, E), which reproduces the speaker's result (B evicted, then C).
- This is Part 1 of the course. Topics such as consistent hashing, replication, message-queue products, rate limiting, WebSockets and idempotency are covered in the companion notes.